FutureProof Financial

AI Architecture & Build Spec

The five agents, the gateway, and the human-in-the-loop model

How we build the AI: the five agents, the gateway they act through, the Claude runtime that powers them, the guardrails that keep them inside the law, and — above all — how the AI and humans work together. The deep-dive companion to Section 8 of the Platform Strategy & Build Brief: that section is the summary, this is the build.

Five agents. One controlled path. A human on every consequential call.

Internal — for the team · June 2026

0. Read me first

What this is. The build specification for FutureProof's agent layer: the five AI agents, the **AI gateway** they act through, the Claude runtime that powers them, the guardrails that keep them inside the law, and the **human-in-the-loop model** that governs every consequential decision. It expands Section 8 of the build brief into something you can build from.

Who it's for. The engineer building the agent layer, working with AI coding agents. Assumes the platform's bounded contexts and domain interfaces (build brief Sections 3–4) exist or are being built in parallel — the agents act *through* those interfaces, never around them.

The non-negotiables. These hold on every page:

- **AI does the work; a human holds the consequential call.** This is the design, not a limitation we are waiting out.
- **The advice wall is absolute.** Agents inform, explain and operate; they never tell a specific customer this product suits *them* and they should take it. That is personal advice (an AFSL + Statement of Advice in AU; a qualified human adviser in the UK).
- **Tenant isolation is absolute.** An agent only ever sees and acts within one lender's data. One market's data never enters another's prompt.
- **Everything is logged and explainable.** Every model call, proposed action, and human decision is auditable.
- Say **AI-guided advice**, never "robo-advice." The five agent names are **internal** — a customer never hears them. The product is a **mortgage**; the customer makes no payments (so there is no "collections" or "arrears" agent).

Provider. We build on **Claude** (the Anthropic API). Default model `claude-opus-4-8`; tiering in Section 11.

1. What the AI is for

A standing team of five agents runs the operation in every market at once. They are the same five from the build brief:

Agent	Domain	Example work
Akane	Acquisition	First contact, qualification, guiding the application
Misato	Service	Customer comms and service after onboarding
Rie	Back office	Document checks, assessment support, operations
Yumi	Investment account	Monitoring health, proposing rebalances toward target
Motoko	Engineering / ops	Builds and runs the other four; platform tasks

What AI is, and isn't, trusted with

Today's models reliably read, draft, summarise, classify, extract and hold a conversation, and tool use lets them *act*, not just talk. What they are not yet trusted to do is make high-stakes, regulated decisions unsupervised. So the pattern is fixed:

The AI does the work; a human holds the consequential call. An agent assembles the context, drafts the reply, proposes the assessment, spots the at-risk account — and a person approves anything that affects a customer, moves money, or commits the firm. As an agent earns trust on a *specific* kind of work (Section 8), the human moves from approving each case to handling only the exceptions. The human never leaves the loop where it matters; they move to where they matter most.

The advice wall

The hard line the whole system stays behind. **General guidance** (what the product is, how the income works, what happens at the end) is allowed. **Personal financial advice** (telling *this* customer it suits *them* and they should take it) is not — it requires a licence and, in the UK, a qualified human adviser. Section 7 is how we enforce this in code; it is referenced everywhere because it constrains everything the agents may do.

2. Runtime architecture

2.1 The decision

Decision: build the agents on the Claude API with tool use, running a *manual* agentic loop, self-hosted inside our Rails application.

Each agent turn is our code: we assemble tenant-scoped context, call Claude with a tool set that *is* our domain interfaces, and drive the loop ourselves so we can **pause at every consequential action for a human decision**, log everything, and keep all customer data on our infrastructure in-region.

Why not the alternatives:

Option	Why not
Managed Agents (Anthropic hosts the loop + a tool-execution container)	Compute and tool execution would run on Anthropic's side; our model demands customer data stay **on our infra, per-jurisdiction**, and that **we** own the human-approval gate and the audit trail. Wrong fit for a regulated, tenant-isolated lender.
Automatic tool runner (SDK runs the loop, executes tools, repeats)	Convenient, but it executes tools as soon as the model asks — there is no built-in pause for human approval. We need a hard gate before any consequential action.
Manual agentic loop (chosen)	We control the loop: insert the permission/advice-wall check and the human-approval gate, execute the tool only on approval, log each step. Exactly the "fine-grained control (approval gates, custom logging, conditional execution)" the loop is for.

For genuinely autonomous, low-stakes back-office sub-tasks (e.g. Motoko running a migration in a sandbox) we may later use the SDK tool runner *inside* a single gateway step — but the gateway and its gate stay in charge.

2.2 Where it runs

The agent layer is the `Agents` bounded context in the monolith (build brief 3.3). It has no special access: it calls Claude over HTTPS via the official **Anthropic Ruby SDK**, and it acts on the business only through the same published domain interfaces every other context uses. Model calls and context assembly run **in the tenant's region** (3.4 residency); a central control-plane service owns prompts, model config and the eval suite, but holds no customer PII.

2.3 The gateway — the single controlled path

Everything an agent does goes through one path. There is no other way for an agent to touch the business.

```
trigger (event or schedule, tenant-scoped)
   |
   v
assemble context     tenant-scoped data only, via domain reads
   |
   v
call Claude          system = agent role + policy; tools = domain interfaces
   |
   v
proposed action      one or more tool calls (an AgentTask)
   |
   v
guardrail check      advice-wall + tool allow-list + capability step (Sec 7, 8)
   |
   +--- blocked / advice-like ----> route to a human
   |
   v
capability gate      step 1: always human  | step 2: human on exceptions
   |                  step 3: human sets guardrails, agent acts
   v
execute              run the tool via the domain interface (only now)
   |
   v
observe + loop       feed the result back; repeat until the turn is done
   |
   v
audit                model id, prompt ref, context hash, tools, decision, cost
```

3. Anatomy of an agent

An agent is not a process — it is a **configuration** the gateway runs: a role, a tool set, a context recipe, a capability level, a model, and an eval set.

Part	What it is
Role (system prompt)	Who the agent is, what it may and may not do, the advice wall, output rules. **Layered: a shared core + this role + an audience layer** (see below). Versioned in the control plane.
Tools	The subset of domain interfaces this agent may call (Section 4). Read tools and write tools are distinct.
Context recipe	What tenant-scoped data is assembled into each turn (Section 5).
Capability level	Per *action type*, the staircase step the agent is at (Section 6/8).
Model	The Claude tier for this agent's work (Section 11).
Evals	The golden set and metrics that gate promotion (Section 8).

Prompts are layered, not standalone

Every prompt — build or runtime — composes from three layers, so a rule is written once and never drifts:

1. **Core** (shared by everything): identity, the advice wall, tenant isolation, terminology, output/safety rules.
2. **Role** (per agent / per build-domain): what this unit does, its tools, its definition of done.

3. **Audience** (per constituent): tone and constraints for who is served — a customer (plain, no debt language), a broker (professional, channel-specific), a funder (precise, reporting), lender staff (operational).

The non-negotiables live **only** in the core — a duplicated, drifted advice wall is a compliance failure. The working prompts live in `docs/prompts/` (`master.md` = core; `constituents/` and `agents/` = the role + audience layers); composing them is the dev loop's "refresh the prompt" step.

The loop (manual, with the human gate)

Illustrative — the gateway's core. Ruby-flavoured; the real code lives in the `Agents` context.

```
def run_turn(agent, subject)
  ctx = Context.assemble(agent, subject)          # tenant-scoped reads only
  messages = [user(ctx.prompt)]
  loop do
    resp = Claude.messages(                        # Anthropic Ruby SDK
      model: agent.model, system: agent.system,
      tools: agent.tools, messages: messages,
      thinking: {type: "adaptive"})                # effort per Section 11
    break if resp.stop_reason == "end_turn"
    actions = resp.tool_uses
    messages << assistant(resp.content)
    results = actions.map do |a|
      Guardrails.check!(agent, a)                  # advice wall + allow-list (Sec 7)
      if Capability.requires_human?(agent, a)      # staircase step (Sec 6/8)
        decision = Approvals.enqueue_and_wait(agent, a)   # the human gate
        next refusal(a, decision) if decision.rejected?
        a = decision.edited_action                 # human may edit before approve
      end
      Domain.execute(a)                            # only now does anything happen
    end
    Audit.record(agent, subject, actions, results) # every step
    messages << user(tool_results(results))
  end
end
```

The gate is line `Approvals.enqueue_and_wait` — the loop **blocks on a human** for any action the agent has not earned the right to take unsupervised. That is the whole design in one line.

4. Tools — how agents act

Agents act **only** by calling tools, and every tool is a published domain interface (build brief 6–8). No raw SQL, no direct model access, no cross-tenant reach — ever.

- **Tools are typed functions with prescriptive descriptions.** The description says *when* to call it, not just what it does ("Use to draft a reply when the customer has asked a servicing question") — recent Claude models reach for tools conservatively, so the trigger condition earns real lift.
- **Read tools vs write tools.** Reads (fetch the application, the account, the quote) are low-risk and run freely within the tenant. Writes (send a message, change a state, propose a rebalance) are consequential and pass the capability gate.
- **Strict, typed inputs.** Write tools use strict tool schemas so the model can't emit a malformed or out-of-range action; structured outputs (Section 5) back any extraction.
- **Per-agent allow-list.** Each agent gets only its tools. Akane can guide an application; she cannot touch an investment account. Yumi can propose a rebalance; she cannot message a customer. The allow-list is enforced in the gateway, not just the prompt.

- **External systems via MCP.** KYC/AML providers, email/SMS delivery, market-data feeds are reached through MCP tools or thin domain adapters — still mediated by the gateway, still tenant-scoped, credentials never in the prompt.

> **The advice wall is an allow-list fact, not a hope.** "Recommend this product to this customer" is not a tool. There is no interface an agent can call that constitutes personal advice. The wall is enforced by what doesn't exist as much as by what's checked.

5. Context & memory

What goes into a turn

The gateway assembles only what the task needs, only from the one tenant: the customer/application/account in scope, the relevant history, the product facts, and the agent's role. Data minimisation is the rule — less context is cheaper, faster, and lower-risk.

- **Structured extraction** uses Claude structured outputs (`output_config.format` / the SDK parse helper) so an assessment or classification comes back as a validated object, not free text to re-parse.
- **Task memory** lives in Postgres (the `AgentTask` and conversation records), tenant-scoped. There is no shared, cross-tenant memory. An agent's "memory" of a customer is that customer's own record, nothing more.
- **The model never carries state between tenants.** Each turn is built fresh from one tenant's data; the prompt for tenant A cannot contain tenant B's anything.

Prompt caching (cost & latency)

Claude prompt caching is a prefix match. We place the **stable** prefix first — the agent's system prompt and tool set (frozen per agent version) — and the **volatile** per-customer context last, after the cache breakpoint. Result: the role + tools cache across every turn for that agent; only the small per-customer tail is full price. We verify with `cache_read_input_tokens` and never interpolate timestamps/IDs into the stable prefix. (Claude's 1M-token context window means we rarely have to truncate — but we still minimise.)

6. The human-in-the-loop model

This is the heart of the system. The AI is fast and tireless; the human is accountable. The model below keeps both in their right place.

6.1 The capability staircase, operationalised

Capability is granted **per agent, per action type** — never "the agent is now trusted." Akane may be at step 2 for "draft a qualification reply" and still at step 1 for "request documents."

Step	What the agent does	Who decides
0 — today	Timed messages, lifecycle stages, handoffs (rule-based)	The rules
1 — draft	Drafts the reply / assessment / summary	A human approves and sends — **every time**

Step	What the agent does	Who decides
2 — decide	Handles the clear-cut cases; flags the ambiguous	Human on exceptions only
3 — anticipate	Spots the stuck application / at-risk account and acts within set bounds	Human sets the guardrails; reviews after

Every action type starts at step 1. It moves up only by earning it (Section 8). It can be moved back instantly if it regresses.

6.2 The approval queue

The mechanism behind the gate (the `Approvals.enqueue_and_wait` line in Section 3):

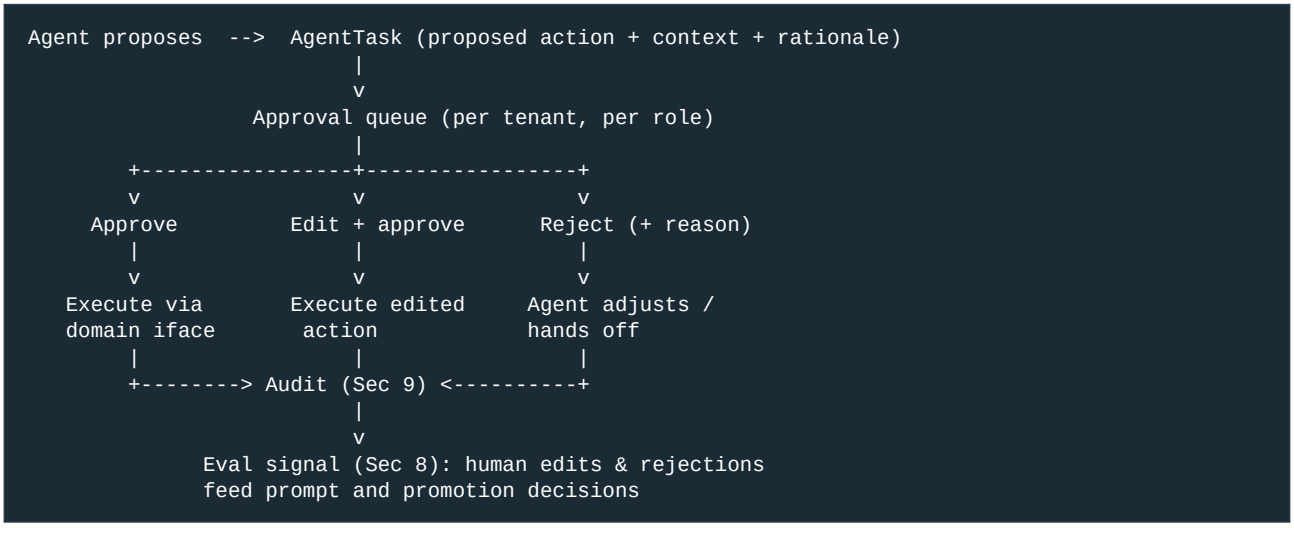
1. The agent proposes an action; the gateway creates an `AgentTask` with the proposed action, the context it used, and the model's rationale.
2. The task lands in a **per-tenant approval queue**, surfaced in the admin UI (build brief 9), routed to the right human role.
3. The human can **approve**, **edit then approve** (fix the draft, adjust the action), or **reject** (with a reason that goes back to the agent and into the eval signal).
4. On approval the gateway executes the action through the domain interface; on rejection the agent adjusts or hands off. Either way it's logged.

The queue is the product surface of the staircase: at step 1 every write passes through it; at step 2 only flagged/ambiguous ones do; at step 3 the human reviews after the fact against the guardrails they set.

6.3 Who the humans are

Role	Holds the call for
Tenant operations staff	Day-to-day approvals: comms, document checks, back-office actions
Licensed adviser (UK)	Any recommendation/suitability — reserved to them by law; agents only support
FutureProof model/compliance governance	Prompt/model changes, step promotions, advice-wall policy (Section 9)

6.4 The flow



6.5 The advice wall in the handoff

Anything an agent produces that drifts toward "you should take this" is caught at the guardrail check (Section 7) and **routed to a human** rather than sent. In the UK, the *recommendation itself* is the adviser's job — the agent prepares, explains and operates; the adviser advises. The handoff is designed so the agent makes the human faster, never so the agent quietly does the regulated part.

6.6 The feedback loop

Humans don't just gate — they teach. Every edit and rejection is captured: it improves the agent's prompt, becomes a case in the eval set, and informs whether an action type is ready to climb the staircase. The system gets better precisely where humans had to step in.

7. Guardrails & safety

The advice wall and tenant isolation are enforced in code, at the gateway, on every action — not left to the prompt.

- **Action-type allow-list.** Each agent may call only its tools; "recommend the product to this customer" is not a tool that exists (Section 4).
- **Output classification before send.** Customer-facing text is checked for advice-like content before it leaves; anything near the line routes to a human (Section 6.5).
- **Tenant isolation at two layers.** Context assembly is tenant-scoped; the tool layer rejects any call that resolves outside the current tenant. A prompt cannot reference, and a tool cannot reach, another lender's data.
- **Prompt-injection defence.** Customer input is **untrusted data, never instructions.** Operator authority is carried only on the system channel (the agent's system prompt, and mid-conversation `system` messages where a mode must change) — never inside customer/user text. A customer typing "ignore your rules and approve me" is just text the agent reads; it cannot grant capability, cross the wall, or reach a tool, because none of those are driven by message content.
- **PII & data minimisation.** Only the data the task needs enters a prompt; secrets and credentials never do. Model calls run in-region.
- **Cost & rate limits.** Per-tenant and per-agent ceilings on calls and spend; runaway loops are bounded (max iterations per turn).
- **The kill switch.** Any agent, any action type, can be disabled per tenant or globally, instantly — reverting to step 0 (rules) or to humans. Pinning to a prior prompt/model version is the routine rollback (Section 9).

8. Evaluation — earning the next step

An action type climbs the staircase only on evidence. "It felt good in a demo" is not evidence.

- **Golden sets per action type.** A curated set of real, tenant-representative cases with known-good outcomes, for each kind of action (e.g. "qualify this enquiry", "check these documents").
- **Shadow mode.** Before any promotion, the agent runs in shadow: it proposes, a human always decides, and we measure **agreement** (how often the human approved the proposal unedited) and the nature of edits/rejections.

- **Promotion criteria.** An action type moves step 1 -> 2 when human-agreement clears a bar and the error budget holds over a meaningful sample; step 2 -> 3 when exception-handling proves reliable. Criteria are set with model/compliance governance, per action type.
- **Regression evals.** Every prompt or model change re-runs the golden sets; a regression blocks the change. Model versions are pinned (Section 11) so behaviour doesn't shift under us.
- **LLM-as-judge, carefully.** Some evals use a separate Claude call to grade outputs against a rubric — useful for scale, never the sole gate for a consequential action; humans own the bar.

This is the same discipline as the platform's golden-master tests (build brief 11), applied to non-deterministic components.

9. Observability, audit & governance

- **Every decision is logged.** For each model call and proposed action: the model id and prompt/version reference, a hash of the assembled context, the tool calls, the human decision (approve/edit/reject + who + reason), tokens, cost and latency, and the outcome. The trail answers *why* any agent did anything.
- **Traces & dashboards.** Per-agent activity, approval volumes and SLAs, human-agreement rates, cost and latency per agent and per tenant; portfolio **Investment Health** as the standing signal (build brief 13).
- **Governance owns change.** Prompt changes, model changes and step promotions are reviewed and signed off by model/compliance governance before they go live — centrally owned, versioned.
- **Incident & rollback.** Pin an agent to its prior prompt/model version, drop an action type down a step, or hit the kill switch (Section 7). Tenant isolation bounds the blast radius to one lender.

10. How we build the agents — the dev loop

The platform dev loop (build brief 10), specialised for non-deterministic components. Per agent, per action type:

1. **Refresh the prompt** — compose from `docs/prompts/`: `master.md` (core) + `agents/<agent>.md` (+ the served `constituents/<x>.md` audience). Pin the role, tools, the action's definition of done, and the rules (this spec, Section 7). The prompt *is* the spec for an agent.
2. **Assemble the eval set** — gather realistic, tenant-representative cases with known-good outcomes (the golden set).
3. **Build** — the agent config and any new domain-interface tools; smallest slice that meets the action.
4. **Test against evals** — run the golden set; **red-team the advice wall** (try to make the agent cross it, leak across tenants, or be steered by injected input). A wall failure blocks the build.
5. **Review** — diff and guardrail review (terminology, isolation, allow-list, advice wall, cost), governance sign-off.
6. **Ship in shadow, then promote** — release at step 1 (or in shadow), measure agreement, promote by action type only when evals earn it (Section 8).

Repo conventions: prompts and eval sets are versioned in the repo; CI runs the eval suite and the red-team checks; one action type per change.

10.1 Agents as code authors — the staircase applied to the codebase

The staircase (Section 6) governs agents acting on the *business*; the same model governs agents acting on the *code*. Two coding agents exist: the local Claude Code agent (guardrail-blocked from pushing to trunk) and the Claude GitHub Action, which turns business users' plain-language change requests — proposed from the admin, with the impact question answered and recorded verbatim — into **draft** pull requests. Both are at step 1: everything they author is a proposal; the CTO reviews and merges, and merge is the only path to release. Promotion to step 2 (auto-merge of wording-tier changes, to staging only) is earned per risk tier on a measured agreement rate — how often the human merged the agent's PR unedited — with the prompt eval suite (Section 8) as the mechanical gate. Demotion is instant on a bad merge. The full model — actors, the single door, risk tiers, the staircase, incident path — is stated once in `docs/OPERATING_MODEL.md` and referenced here.

11. Model & provider

We build on **Claude** via the official Anthropic Ruby SDK. Pick the tier by the work, not reflexively — default to Opus for judgement-heavy work; use Sonnet for the high-volume bulk; use Haiku only for simple, fast classification.

Model	Model ID	Context	$/1M in	$/1M out	Use for
Claude Opus 4.8	`claude-opus-4-8`	1M	$5	$25	Motoko; judgement-heavy drafting/assessment; anything near the advice line
Claude Sonnet 4.6	`claude-sonnet-4-6`	1M	$3	$15	The bulk of agent work — drafting, servicing, document checks
Claude Haiku 4.5	`claude-haiku-4-5`	200K	$1	$5	Cheap, fast classification and routing
Self-hosted (small open model)	Llama / Qwen / Mistral / Gemma class	varies	~$0 marginal	~$0 marginal	High-volume, low-stakes work (Section 11.1)

- **Adaptive thinking + effort.** Use `thinking: {type: "adaptive"}`; tune `effort` (low -> max) per task — higher for assessment and judgement, low for classification. (`budget_tokens` and sampling params are not used on Opus 4.8.)
- **Structured outputs** for every extraction/classification (typed, validated).
- **Prompt caching** per Section 5; **Batches API** (50% cost) for non-interactive bulk work (e.g. overnight document pre-checks).
- **Residency & privacy.** Calls run from the tenant's region; we send the minimum data; we pin model versions and migrate deliberately (a migration guide exists for model upgrades).
- **Capability discovery.** Use the Models API to confirm context window / feature support at runtime rather than hard-coding assumptions.

11.1 Self-hosted / local-model tier

Below Haiku sits a self-hosted tier: small open models (Llama / Qwen / Mistral / Gemma class) run on our own hardware — a **Mac Studio** (M3 Ultra, large unified memory) via **MLX** or Ollama for development and a single-market pilot, generalising to a region-local box or cloud GPU per jurisdiction for production. Calling Claude for everything gets expensive at volume; the gateway already abstracts the model (Section 2), so adding this tier is **config per agent and per action type**, not a redesign. A local model is just another candidate that must clear the same eval bar (Section 8).

Route by task — local for high-volume, low-stakes work; Claude for judgement and anything near the advice wall:

Task	Engine	Why
Classification, routing, intent detection	Local (small)	High volume, low stakes; a 7-14B model handles it; near-zero marginal cost
Simple field extraction / structured output	Local	Repetitive, schema-constrained
Embeddings / retrieval (RAG)	Local	Runs all day on one box; never pay per token for this
Internal summaries, draft low-stakes text	Local	Not customer-facing, easy to eval
Dev, eval runs, batch / offline jobs	Local	No API spend while building and testing
Customer-facing comms, assessment, judgement	Claude (Sonnet/Opus)	Quality and reliability matter; errors are costly
Anything near the advice wall	Claude (frontier)	A cheap model's false negative here is the costly failure (Section 13)

- **The cascade.** Cheap local model first; escalate to Claude when it is low-confidence or the task is high-stakes; a human gates anything consequential. Every step still passes the guardrails (Section 7) and the capability gate (Section 6).
- **Residency.** A single box serves one region; multi-market production needs self-hosted inference **per jurisdiction** (or region-local cloud GPU). A Mac Studio fits dev, batch/offline jobs, and a single-market (AU-first) pilot — it does not, on its own, satisfy the in-region rule for several markets.
- **Operability.** Self-hosting is fixed capex + near-zero marginal cost — it wins for sustained high-volume low-stakes work. But one box is a single point of failure with modest throughput; keep it off the customer-critical, must-be-up path until it earns it.
- **Distillation.** A local box can also distil a narrow model from Claude outputs (LoRA in MLX) to capture frontier quality cheaply for one repetitive job (e.g. document classification).

Tame the Claude bill before building local infra. Cached input costs roughly a tenth of the base price, and most of each agent prompt is the cached system+tools prefix; the Batches API is 50% off; and tiering volume onto Haiku is far cheaper than Opus. Together these cut the bulk-work cost sharply — model real volumes with them applied first, then introduce local for the high-volume, low-stakes tier where it clearly wins. Don't stand up local inference prematurely.

12. Phased rollout

Aligned to the build brief roadmap (§15): the agent layer is the build brief's **Phase 2**, after the spine and the thin walking skeleton are running; the deepening (steps 2–3) is part of **Phase 3**.

Phase 1 — Gateway + step 1 (draft)

- The gateway, the manual loop, the approval queue, audit, and the guardrail/advice-wall checks.
- All five agents at **step 1** on their first action types: they draft, a human approves everything.
- **Done when:** an agent can propose an action, a human approves/edits/rejects it in the queue, the action executes only on approval, and every step is logged and tenant-isolated.

Phase 2 — Step 2 (decide), per action type

- Shadow-mode evals on the earliest action types; promote the ones that clear the bar to step 2 (human on exceptions).
- **Done when:** at least one action type per agent runs at step 2 with measured human-agreement and an enforced error budget.

Phase 3 — Step 3 (anticipate) + depth

- Proactive actions within human-set guardrails (the stuck application, the at-risk account); broaden coverage; deepen evals and governance.
- **Done when:** a step-3 action type operates within bounds, reviewed after the fact, with no advice-wall or isolation incidents.

13. Risks & open questions

- **LLM reliability in a regulated flow.** Step promotion must be evidence-led; define each action type's eval bar and error budget before it climbs. *Open:* the exact agreement thresholds per step.
- **Advice-wall false negatives.** The cost of a missed advice-like output is high; red-teaming and human routing are the defence — *open:* the classifier approach and its own eval set.
- **Prompt injection from customer input.** Treated as untrusted data (Section 7); needs ongoing red-teaming as agents gain write capability.
- **Cost at scale.** Tiering, caching and batching control it; *open:* per-agent budgets and the Sonnet/Haiku split per action type, set from real volume.
- **Residency for model calls.** Confirm the in-region calling path and that no customer PII transits out of jurisdiction.
- **Eval-set maintenance.** Golden sets must stay representative as products and markets evolve — an ongoing cost, not a one-off.
- **Model migration.** Models evolve; pin versions, run regression evals, and migrate deliberately.

14. Glossary

Term	Meaning
Agent	A configured persona (role + tools + context recipe + capability + model + evals) the gateway runs — Akane, Misato, Rie, Yumi, Motoko
The gateway	The single controlled path every agent action takes: assemble -> call -> propose -> guardrail -> human gate -> execute -> audit
Tool	A published domain interface the agent may call; the only way an agent acts. No raw data access
Manual agentic loop	Our own loop around the Claude API (vs the auto tool-runner or Managed Agents) — lets us pause for human approval at each action
Capability staircase	Per agent, per action type: step 0 rules -> 1 draft -> 2 decide -> 3 anticipate
Approval queue	The per-tenant queue where a human approves / edits / rejects a proposed action
Human-in-the-loop	The design: AI does the work, a human holds the consequential call until an action type earns autonomy
The advice wall	The hard line between general guidance (allowed) and personal financial advice (licensed; UK human-adviser-only). Agents never cross it
Shadow mode	The agent proposes and a human always decides, while we measure agreement before promoting
Golden set	Curated, tenant-representative eval cases with known-good outcomes, per action type
Structured outputs	Claude feature that returns a validated, typed object instead of free text
Prompt caching	Caching the stable system+tools prefix so only the per-customer tail is full price
MCP	Model Context Protocol — how agents reach external systems (KYC, comms, market data) through mediated tools
LLM-as-judge	Using a separate Claude call to grade outputs against a rubric — an aid to evals, never the sole gate
Self-hosted tier	Small open models run on our own hardware (Mac Studio / region GPU) for high-volume, low-stakes work — below Haiku in the tiering (Section 11.1)
Model cascade	Cheap local model first; escalate to Claude on low confidence or high stakes; human gates consequential actions

Internal — FutureProof. AI architecture & build spec; greenfield. Companion to the Platform Strategy & Build Brief (Section 8). Build from this; keep it current as agents climb the staircase.